Guia de Estudio para Entrevistas Tech

Preparado para Lucas Vicente — Full-Stack Developer & DevOps
Plan de estudio: 2 semanas · Basado en tu CV y stack real
Objetivo: Dominar los conceptos que te van a preguntar en entrevistas

> **Regla de oro:** No necesitas memorizar codigo. Necesitas ENTENDER que hace cada cosa, POR QUE se usa y CUANDO elegirla. Si puedes explicarlo con tus palabras, pasas la entrevista.

Plan de 2 Semanas

SEMANA 1 Fundamentos (lo que te preguntan seguro)

- Dia 1-2: JavaScript esencial (variables, funciones, arrays, promesas)
- Dia 3: TypeScript basico (tipos, interfaces)
- Dia 4-5: React (componentes, hooks, estado, props)
- Dia 6: Next.js (routing, SSR vs CSR, API routes)
- Dia 7: HTML/CSS fundamentals + repaso

SEMANA 2 Backend, DevOps y entrevista

- Dia 8: Node.js y APIs REST
- Dia 9: SQL y PostgreSQL
- Dia 10: Docker (conceptos, comandos, Dockerfile)
- Dia 11: Linux, redes y Git
- Dia 12: AWS Cloud - conceptos y servicios esenciales
- Dia 13: Terraform - infraestructura como codigo
- Dia 14: Preguntas de comportamiento, vender proyectos + tests finales

Indice de Contenidos

JavaScript Esencial
TypeScript Basico
React Fundamentals
Next.js
HTML y CSS
Node.js y APIs REST
SQL y PostgreSQL
Docker
Linux y Redes
Git
Conceptos Clave de Arquitectura
Preguntas de Comportamiento
Como Vender tus Proyectos
AWS Cloud Esencial
Terraform - Infraestructura como Codigo
100+ Preguntas Frecuentes de Entrevista

1. JavaScript Esencial

JavaScript es la base de todo tu stack. Te van a preguntar sobre esto en el 95% de las entrevistas frontend/fullstack.

1.1 Variables: var, let, const

Keyword	Scope	Reasignable	Cuando usarlo
`var`	Funcion	Si	NUNCA (legacy)
`let`	Bloque	Si	Cuando el valor cambia
`const`	Bloque	No	Por defecto siempre

> **Pregunta tipica:** "Cual es la diferencia entre let y const?" — const no se puede reasignar, pero si es un objeto/array, sus propiedades SI se pueden mutar.

```
const user = { name: "Lucas" };
user.name = "Pedro";  // OK, mutas la propiedad
user = {};            // ERROR, no puedes reasignar
```

1.2 Tipos de datos

- **Primitivos:** string, number, boolean, null, undefined, symbol, bigint
- **Referencia:** object, array, function

> **Pregunta:** "Que diferencia hay entre null y undefined?"
> `undefined` = variable declarada pero sin valor asignado.
> `null` = valor asignado intencionalmente para indicar "vacio".

1.3 Funciones

```
// Funcion clasica
function sumar(a, b) {
  return a + b;
}

// Arrow function (la que mas usaras en React)
const sumar = (a, b) => a + b;

// Arrow con cuerpo
const procesar = (data) => {
  const resultado = data.filter(x => x > 0);
  return resultado;
};
```

> **Diferencia clave:** Las arrow functions NO tienen su propio `this`. Heredan el `this` del contexto donde se definen. Las funciones clasicas SI tienen su propio `this`.

1.4 Arrays - Metodos que DEBES saber

Estos los vas a usar todos los dias en React:

```
const nums = [1, 2, 3, 4, 5];

// MAP - transforma cada elemento, devuelve nuevo array
nums.map(n => n * 2);           // [2, 4, 6, 8, 10]

// FILTER - filtra elementos, devuelve nuevo array
nums.filter(n => n > 3);        // [4, 5]

// FIND - devuelve el PRIMER elemento que cumple
nums.find(n => n > 3);          // 4

// REDUCE - acumula valores
nums.reduce((acc, n) => acc + n, 0);  // 15

// SOME - al menos uno cumple? (boolean)
nums.some(n => n > 4);          // true

// EVERY - todos cumplen? (boolean)
nums.every(n => n > 0);         // true

// INCLUDES - contiene el valor?
nums.includes(3);               // true

// forEach - ejecuta funcion por cada elemento (NO devuelve nada)
nums.forEach(n => console.log(n));
```

CLAVE: `map`, `filter` y `reduce` NO mutan el array original. Devuelven uno nuevo. Esto es fundamental en React donde la inmutabilidad importa.

1.5 Destructuring

```
// Objetos
const user = { name: "Lucas", age: 25, city: "Madrid" };
const { name, age } = user;     // name = "Lucas", age = 25

// Arrays
const [first, second] = [10, 20];  // first = 10, second = 20

// Con valores por defecto
const { role = "dev" } = user;  // role = "dev" (no existia)

// Renombrar
const { name: nombre } = user;  // nombre = "Lucas"
```

1.6 Spread y Rest

```
// SPREAD - expandir
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4];   // [1, 2, 3, 4]

const obj1 = { a: 1 };
```

```
const obj2 = { ...obj1, b: 2 }; // { a: 1, b: 2 }

// REST - agrupar el "resto"
const { name, ...rest } = user; // rest = { age: 25, city: "Madrid" }

function sum(...nums) {          // nums = [1, 2, 3]
  return nums.reduce((a, b) => a + b, 0);
}
```

1.7 Promesas y Async/Await

Esto es CRITICO. Toda comunicacion con APIs, bases de datos, etc. es asincrona.

```
// Promesa basica
fetch("https://api.ejemplo.com/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));

// Async/Await (la forma moderna - usa esta)
async function getUsers() {
  try {
    const response = await fetch("https://api.ejemplo.com/users");
    const data = await response.json();
    return data;
  } catch (error) {
    console.error("Error:", error);
  }
}
```

> **Pregunta tipica:** "Que es una promesa?"
> Un objeto que representa un valor que puede estar disponible ahora, en el futuro, o nunca. Tiene 3 estados: **pending** (esperando), **fulfilled** (resuelta con valor), **rejected** (rechazada con error).

1.8 Operador Ternario y Optional Chaining

```
// Ternario (lo usas MUCHO en JSX)
const msg = age >= 18 ? "Mayor" : "Menor";

// Optional chaining - evita errores si algo es null/undefined
const city = user?.address?.city;  // undefined si algo no existe (no explota)

// Nullish coalescing - valor por defecto solo si null/undefined
const name = user.name ?? "Anonimo";
```

1.9 Closures

```
function crearContador() {
  let count = 0;              // esta variable "vive" dentro del closure
  return function() {
    count++;
    return count;
```

```
  };
}

const contador = crearContador();
contador(); // 1
contador(); // 2
contador(); // 3
```

Pregunta: "Que es un closure?"
Una funcion que recuerda las variables del scope donde fue creada, incluso cuando se ejecuta fuera de ese scope. Es como si la funcion "capturara" su entorno.

1.10 Event Loop

Pregunta: "Como funciona el Event Loop en JavaScript?"
JS es single-threaded. El Event Loop es el mecanismo que permite ejecutar codigo asincrono sin bloquear el hilo principal:

1. **Call Stack** - donde se ejecuta el codigo sincrono
2. **Web APIs** - donde se procesan las operaciones async (fetch, setTimeout, etc.)
3. **Callback Queue** - cola de funciones listas para ejecutarse
4. **Event Loop** - revisa si el Call Stack esta vacio; si lo esta, mueve callbacks de la queue al stack

Por eso un `setTimeout(fn, 0)` no se ejecuta inmediatamente: primero va a la queue y espera a que el stack se vacie.

2. TypeScript Basico

TypeScript es JavaScript con tipos. Lo usas en todos tus proyectos. Te van a preguntar al menos 1-2 preguntas sobre esto.

2.1 Tipos basicos

```
// Tipos primitivos
let nombre: string = "Lucas";
let edad: number = 25;
let activo: boolean = true;

// Arrays
let nums: number[] = [1, 2, 3];
let tags: Array<string> = ["react", "next"];

// Objetos
let user: { name: string; age: number } = { name: "Lucas", age: 25 };

// Union types - puede ser uno u otro
let id: string | number = "abc";
id = 123;  // tambien valido
```

2.2 Interfaces vs Types

```
// Interface - para objetos (preferida para definir "formas")
interface User {
  id: number;
  name: string;
  email: string;
  role?: string;        // ? = opcional
  readonly createdAt: Date;  // no se puede modificar
}

// Type - mas flexible (unions, intersections, primitivos)
type Status = "active" | "inactive" | "banned";  // union literal
type ID = string | number;

// Extender
interface Admin extends User {
  permissions: string[];
}

// Intersection con types
type AdminUser = User & { permissions: string[] };
```

> **Pregunta:** "Cuando usas interface y cuando type?"
> **Interface**: para definir la forma de objetos y clases. Se pueden extender y mergear.
> **Type**: para unions, intersections, tipos primitivos con alias, tipos condicionales. Mas flexible pero no se puede mergear.

2.3 Generics

```
// Funcion generica - funciona con cualquier tipo
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}

firstElement<number>([1, 2, 3]);     // 1 (tipo number)
firstElement<string>(["a", "b"]);    // "a" (tipo string)

// Interface generica
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

// Uso
const response: ApiResponse<User[]> = {
  data: [{ id: 1, name: "Lucas", email: "...", createdAt: new Date() }],
  status: 200,
  message: "OK"
};
```

> **Explicacion simple:** Los generics son como "huecos" que rellenas con un tipo cuando los usas. Es como decir "esta funcion funciona con CUALQUIER tipo, tu me dices cual cuando la llames".

2.4 Utility Types (los mas comunes)

```
// Partial - hace todas las propiedades opcionales
type UpdateUser = Partial<User>;  // todo opcional

// Pick - selecciona solo algunas propiedades
type UserPreview = Pick<User, "name" | "email">;

// Omit - excluye propiedades
type UserWithoutId = Omit<User, "id">;

// Record - crea un tipo de objeto con claves y valores tipados
type Roles = Record<string, string[]>;  // { [key: string]: string[] }
```

3. React Fundamentals

React es el core de tu stack frontend. Esto te lo van a preguntar SI O SI.

3.1 Que es React

> **Pregunta:** "Que es React y por que se usa?"
> Es una libreria de JavaScript para construir interfaces de usuario. Usa un **Virtual DOM** para actualizar solo las partes de la pagina que cambian, en vez de recargar todo. Es declarativo: tu dices QUE quieres ver, React se encarga del COMO.

3.2 Componentes

```
// Componente funcional (el que se usa hoy)
function UserCard({ name, email }: { name: string; email: string }) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>{email}</p>
    </div>
  );
}

// Uso
<UserCard name="Lucas" email="contacto@lucasvicente.es" />
```

3.3 Props vs State

Props	State
Datos que vienen del padre	Datos internos del componente
Solo lectura (inmutables)	Se pueden cambiar con setState
Fluyen de arriba a abajo	Solo existen dentro del componente

3.4 Hooks esenciales

useState - manejar estado

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);  // valor inicial: 0

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(prev => prev - 1)}>-1</button>
    </div>
  );
}
```

Clave: Usa la forma `prev => prev + 1` cuando el nuevo estado depende del anterior. Asi evitas bugs por batching.

useEffect - efectos secundarios

```
import { useEffect, useState } from "react";

function UserList() {
  const [users, setUsers] = useState([]);

  // Se ejecuta al montar el componente (array vacio = solo una vez)
  useEffect(() => {
    async function fetchUsers() {
      const res = await fetch("/api/users");
      const data = await res.json();
      setUsers(data);
    }
    fetchUsers();
  }, []);  // [] = solo al montar

  // Se ejecuta cuando cambia userId
  useEffect(() => {
    console.log("userId cambio");
  }, [userId]);  // se re-ejecuta si userId cambia

  // Cleanup (limpiar al desmontar)
  useEffect(() => {
    const interval = setInterval(() => console.log("tick"), 1000);
    return () => clearInterval(interval);  // cleanup
  }, []);
}
```

Pregunta: "Que es el array de dependencias de useEffect?"
Es el segundo argumento. Le dice a React CUANDO re-ejecutar el efecto:
- `[]` = solo al montar (una vez)
- `[var1, var2]` = cuando var1 o var2 cambien
- Sin array = en CADA render (evitar esto)

useRef - referencia que persiste entre renders

```
const inputRef = useRef<HTMLInputElement>(null);

// Acceder al DOM directamente
inputRef.current?.focus();

// Tambien para guardar valores sin causar re-render
const renderCount = useRef(0);
renderCount.current++;  // no causa re-render
```

useMemo y useCallback - optimizacion

```
// useMemo - memoriza un VALOR calculado
const filteredUsers = useMemo(() => {
```

```
  return users.filter(u => u.active);
}, [users]);  // solo recalcula si users cambia

// useCallback - memoriza una FUNCION
const handleClick = useCallback(() => {
  console.log("click");
}, []);  // misma referencia entre renders
```

No abuses: useMemo y useCallback son optimizaciones. No los uses en todo. Solo cuando tienes calculos costosos o pasas funciones como props a componentes memoizados.

3.5 Renderizado condicional

```
// Con ternario
{isLoggedIn ? <Dashboard /> : <Login />}

// Con && (short-circuit)
{error && <ErrorMessage msg={error} />}

// Con early return
function Page({ user }) {
  if (!user) return <p>Cargando...</p>;
  return <Profile user={user} />;
}
```

3.6 Listas y Keys

```
{users.map(user => (
  <UserCard key={user.id} name={user.name} />
))}
```

Pregunta: "Por que se necesita key en las listas?"
React usa las keys para identificar que elementos cambiaron, se agregaron o se eliminaron. Sin keys unicas, React no puede optimizar el re-renderizado y puede causar bugs visuales. NUNCA uses el indice del array como key si la lista puede cambiar de orden.

3.7 Manejo de formularios

```
function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();  // evita recarga de pagina
    console.log({ email, password });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
```

```
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

3.8 Context API

```
// 1. Crear contexto
const ThemeContext = createContext<"light" | "dark">("light");

// 2. Provider (envuelve componentes que necesitan acceso)
function App() {
  const [theme, setTheme] = useState<"light" | "dark">("light");
  return (
    <ThemeContext.Provider value={theme}>
      <Page />
    </ThemeContext.Provider>
  );
}

// 3. Consumir con useContext
function Button() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Click</button>;
}
```

> **Cuando usar Context:** Para datos "globales" como tema, usuario logueado, idioma. NO para estado complejo ni para datos que cambian muy frecuentemente (usa Zustand o Redux para eso).

4. Next.js

Next.js es el framework que usas sobre React. Anade routing, SSR, API routes y mas.

4.1 Que es Next.js

Pregunta: "Que ventajas tiene Next.js sobre React puro?"
- **File-based routing**: las carpetas y archivos definen las rutas, no necesitas react-router
- **Server-Side Rendering (SSR)**: la pagina se renderiza en el servidor, mejor SEO y carga inicial
- **API Routes**: puedes crear endpoints backend dentro del mismo proyecto
- **Optimizaciones**: imagenes, fonts, code splitting automatico
- **Server Components** (App Router): componentes que se ejecutan en el servidor y no envian JS al cliente

4.2 App Router vs Pages Router

Pages Router (legacy)	App Router (actual)
Carpeta `/pages`	Carpeta `/app`
`getServerSideProps`	Server Components por defecto
`getStaticProps`	`fetch` con cache options
Todo es Client Component	Server Components + "use client"

4.3 Server Components vs Client Components

```
// Server Component (por defecto en App Router)
// Se ejecuta en el servidor, NO envia JS al navegador
async function UserList() {
  const users = await db.query("SELECT * FROM users");  // acceso directo a DB
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// Client Component (necesita interactividad)
"use client";  // esta directiva lo hace client component
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Regla: Usa Server Components por defecto. Solo anade `"use client"` cuando necesites: useState, useEffect, onClick, onChange, o cualquier interactividad del navegador.

4.4 Routing (App Router)

```
app/
  page.tsx          ->  /
  about/
    page.tsx        ->  /about
  blog/
```

```
    page.tsx        ->  /blog
    [slug]/
      page.tsx      ->  /blog/mi-articulo  (dinamico)
  api/
    users/
      route.ts      ->  GET/POST /api/users
```

4.5 API Routes

```
// app/api/users/route.ts
import { NextResponse } from "next/server";

export async function GET() {
  const users = await db.query("SELECT * FROM users");
  return NextResponse.json(users);
}

export async function POST(request: Request) {
  const body = await request.json();
  const user = await db.query("INSERT INTO users ...", body);
  return NextResponse.json(user, { status: 201 });
}
```

4.6 SSR, SSG, ISR

Pregunta: "Diferencia entre SSR, SSG e ISR?"
- **SSR (Server-Side Rendering)**: la pagina se genera en cada peticion. Datos siempre frescos pero mas lento.
- **SSG (Static Site Generation)**: la pagina se genera en build time. Muy rapido pero datos estaticos.
- **ISR (Incremental Static Regeneration)**: SSG pero se regenera cada X segundos. Lo mejor de ambos mundos.

5. HTML y CSS

5.1 HTML Semantico

```
<header>   - cabecera de pagina/seccion
<nav>      - navegacion
<main>     - contenido principal (solo 1 por pagina)
<section>  - seccion tematica
<article>  - contenido independiente
<aside>    - contenido lateral/relacionado
<footer>   - pie de pagina
```

Por que importa: Accesibilidad (lectores de pantalla), SEO (Google entiende la estructura) y mantenibilidad del codigo.

5.2 Flexbox (lo usas constantemente)

```
.container {
  display: flex;
  justify-content: center;    /* eje horizontal */
  align-items: center;        /* eje vertical */
  gap: 16px;                  /* espacio entre items */
  flex-wrap: wrap;            /* items saltan de linea si no caben */
  flex-direction: row;        /* row | column */
}
```

5.3 CSS Grid

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);  /* 3 columnas iguales */
  grid-template-columns: 200px 1fr 1fr;   /* 1 fija + 2 flexibles */
  gap: 16px;
}
```

Pregunta: "Cuando usas Flexbox y cuando Grid?"
Flexbox: layouts en UNA dimension (fila O columna). Navbars, centrar cosas, distribuir items.
Grid: layouts en DOS dimensiones (filas Y columnas). Layouts de pagina completa, galerias, dashboards.

5.4 Box Model

```
/* Cada elemento tiene: content + padding + border + margin */
.box {
  width: 200px;
  padding: 20px;      /* espacio DENTRO del borde */
  border: 1px solid;  /* el borde */
  margin: 10px;       /* espacio FUERA del borde */
  box-sizing: border-box;  /* width incluye padding+border */
}
```

5.5 Responsive Design

```
/* Mobile first: disenar para movil y escalar hacia arriba */
.card { width: 100%; }

@media (min-width: 768px) {   /* tablet */
  .card { width: 50%; }
}

@media (min-width: 1024px) {  /* desktop */
  .card { width: 33.33%; }
}
```

5.6 Tailwind CSS (lo que usas)

```
<!-- Equivalente a escribir CSS manualmente -->
<div class="flex items-center justify-between p-4 bg-white rounded-lg shadow-md">
  <h2 class="text-xl font-bold text-gray-900">Titulo</h2>
  <button class="px-4 py-2 bg-blue-600 text-white rounded hover:bg-blue-700">
    Click
  </button>
</div>
```

> **Si te preguntan:** "Tailwind usa clases de utilidad en vez de escribir CSS custom. Cada clase hace UNA cosa. Es mas rapido de desarrollar, el CSS final es mas pequeno (purge) y evita conflictos de nombres."

6. Node.js y APIs REST

6.1 Que es Node.js

Pregunta: "Que es Node.js?"
Es un entorno de ejecucion de JavaScript fuera del navegador, construido sobre el motor V8 de Chrome. Permite usar JS en el servidor. Es single-threaded pero usa un modelo de I/O no bloqueante (event loop) para manejar muchas conexiones simultaneas.

6.2 REST API - Metodos HTTP

Metodo	Accion	Ejemplo	Body
`GET`	Leer	`GET /api/users`	No
`POST`	Crear	`POST /api/users`	Si
`PUT`	Reemplazar completo	`PUT /api/users/1`	Si
`PATCH`	Actualizar parcial	`PATCH /api/users/1`	Si
`DELETE`	Eliminar	`DELETE /api/users/1`	No

6.3 Codigos de estado HTTP

Codigo	Significado	Cuando
`200`	OK	Todo bien
`201`	Created	Recurso creado (POST exitoso)
`204`	No Content	Exito sin body (DELETE exitoso)
`400`	Bad Request	Datos del cliente invalidos
`401`	Unauthorized	No autenticado
`403`	Forbidden	Autenticado pero sin permisos
`404`	Not Found	Recurso no existe
`500`	Internal Server Error	Error del servidor

Pregunta: "Diferencia entre 401 y 403?"
401: No se quien eres (no estas logueado).
403: Se quien eres, pero no tienes permiso para esto.

6.4 Autenticacion: JWT vs Sessions

JWT (JSON Web Token)	Sessions
Token almacenado en el cliente	ID almacenado en cookie, datos en servidor
Stateless (servidor no guarda nada)	Stateful (servidor guarda la sesion)
Escalable (no necesita estado compartido)	Necesita storage compartido en multiples servidores
No se puede invalidar facilmente	Facil de invalidar (borrar del servidor)

6.5 CORS

> **Pregunta:** "Que es CORS?"
> Cross-Origin Resource Sharing. Es una politica de seguridad del navegador que bloquea peticiones HTTP a un dominio diferente al de la pagina. Si tu frontend esta en `localhost:3000` y tu API en `localhost:8000`, el navegador bloquea la peticion a menos que el servidor envie las cabeceras CORS correctas (`Access-Control-Allow-Origin`).

7. SQL y PostgreSQL

Lo usas en casi todos tus proyectos. Saber SQL basico es obligatorio.

7.1 Queries basicas

```
-- Seleccionar todo
SELECT * FROM users;

-- Seleccionar columnas especificas con condicion
SELECT name, email FROM users WHERE active = true;

-- Ordenar
SELECT * FROM users ORDER BY created_at DESC;

-- Limitar resultados
SELECT * FROM users LIMIT 10 OFFSET 20;  -- pagina 3 de 10 items

-- Contar
SELECT COUNT(*) FROM users WHERE role = 'admin';

-- Insertar
INSERT INTO users (name, email, role)
VALUES ('Lucas', 'contacto@lucasvicente.es', 'dev');

-- Actualizar
UPDATE users SET role = 'admin' WHERE id = 1;

-- Eliminar
DELETE FROM users WHERE id = 1;
```

7.2 JOINs

```
-- INNER JOIN: solo filas que coinciden en AMBAS tablas
SELECT u.name, o.total
FROM users u
INNER JOIN orders o ON u.id = o.user_id;

-- LEFT JOIN: todas las filas de la izquierda + coincidencias de la derecha
SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
-- Usuarios SIN pedidos tambien aparecen (con o.total = NULL)
```

> **Pregunta:** "Diferencia entre INNER JOIN y LEFT JOIN?"
> **INNER JOIN**: solo devuelve filas donde hay coincidencia en ambas tablas.
> **LEFT JOIN**: devuelve TODAS las filas de la tabla izquierda, aunque no tengan coincidencia en la derecha (rellena con NULL).

7.3 GROUP BY y funciones de agregacion

```
-- Cuantos pedidos tiene cada usuario
SELECT user_id, COUNT(*) as total_orders, SUM(total) as total_spent
FROM orders
GROUP BY user_id
HAVING COUNT(*) > 5;  -- solo los que tienen mas de 5 pedidos
```

7.4 Indices

Pregunta: "Que es un indice y cuando lo usas?"
Un indice es como el indice de un libro: permite encontrar datos rapido sin recorrer toda la tabla. Crealos en columnas que uses frecuentemente en WHERE, JOIN o ORDER BY. Desventaja: ocupan espacio y hacen los INSERT/UPDATE mas lentos.

```
CREATE INDEX idx_users_email ON users(email);
```

7.5 Transacciones

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;  -- si todo OK, guarda los cambios
-- Si algo falla: ROLLBACK; deshace todo
```

Explicacion: Las transacciones garantizan que un grupo de operaciones se ejecuten TODAS o NINGUNA. Si transfieres dinero, no quieres que se reste de una cuenta pero no se sume a la otra.

8. Docker

Docker esta en tu CV en practicamente todos los proyectos. Te van a preguntar.

8.1 Conceptos clave

> **Pregunta:** "Que es Docker y por que se usa?"
> Docker permite empaquetar una aplicacion con TODAS sus dependencias en un contenedor aislado. Ese contenedor funciona igual en tu portatil, en el servidor, en produccion. Elimina el "en mi maquina funciona".

Concepto	Que es	Analogia
Imagen	Plantilla de solo lectura	La receta de un plato
Contenedor	Instancia en ejecucion de una imagen	El plato ya cocinado
Dockerfile	Instrucciones para crear una imagen	Los pasos de la receta
Volume	Almacenamiento persistente	El frigorifico (datos que sobreviven)
Network	Red virtual entre contenedores	La mesa donde se sientan juntos

8.2 Dockerfile basico

```
# Imagen base
FROM node:20-alpine

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar archivos de dependencias primero (cache de capas)
COPY package*.json ./
RUN npm install

# Copiar el resto del codigo
COPY . .

# Construir la aplicacion
RUN npm run build

# Puerto que expone el contenedor
EXPOSE 3000

# Comando para arrancar
CMD ["npm", "start"]
```

> **Por que COPY package.json primero?** Docker cachea cada linea (capa). Si copias todo junto, cualquier cambio en el codigo invalida la cache del npm install. Separandolo, el install solo se re-ejecuta si package.json cambia.

8.3 Docker Compose

```
# docker-compose.yml
version: "3.8"
services:
```

```
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=mydb

volumes:
  pgdata:
```

8.4 Comandos esenciales

```
docker build -t mi-app .              # Construir imagen
docker run -p 3000:3000 mi-app        # Ejecutar contenedor
docker ps                             # Ver contenedores activos
docker ps -a                          # Ver todos (incluidos parados)
docker logs mi-contenedor             # Ver logs
docker exec -it mi-contenedor sh      # Entrar al contenedor
docker stop mi-contenedor             # Parar
docker rm mi-contenedor               # Eliminar contenedor
docker rmi mi-imagen                  # Eliminar imagen
docker-compose up -d                  # Levantar todo en background
docker-compose down                   # Parar y eliminar todo
```

8.5 Docker vs Maquinas Virtuales

> **Pregunta:** "Diferencia entre un contenedor y una VM?"
> **VM**: incluye un SO completo. Pesada (GBs), lenta de arrancar (minutos).
> **Contenedor**: comparte el kernel del host. Ligero (MBs), arranca en segundos.
> Los contenedores son mas eficientes pero menos aislados que las VMs.

9. Linux y Redes

9.1 Comandos Linux esenciales

```
# Navegacion
ls -la                    # listar archivos (incluidos ocultos)
cd /var/log               # cambiar directorio
pwd                       # directorio actual
find / -name "*.log"      # buscar archivos

# Archivos
cat archivo.txt           # ver contenido
tail -f /var/log/syslog   # ver ultimas lineas en tiempo real
grep "error" archivo.log  # buscar texto en archivo
chmod 755 script.sh       # cambiar permisos
chown user:group file     # cambiar propietario

# Procesos
ps aux                    # ver procesos
top / htop                # monitor de recursos en tiempo real
kill -9 PID               # matar proceso
systemctl status nginx    # ver estado de un servicio
systemctl restart nginx   # reiniciar servicio

# Red
curl https://api.com/test      # hacer peticion HTTP
ping google.com                # comprobar conectividad
netstat -tlnp                  # ver puertos abiertos
ss -tlnp                       # alternativa moderna a netstat

# Disco
df -h                     # espacio en disco
du -sh /var/log           # tamano de carpeta
```

9.2 Permisos Linux

```
# rwxrwxrwx = propietario | grupo | otros
# r=read(4) w=write(2) x=execute(1)
chmod 755 file    # propietario: rwx, grupo: r-x, otros: r-x
chmod 644 file    # propietario: rw-, grupo: r--, otros: r--
```

9.3 Redes - Conceptos clave

> **Pregunta:** "Que pasa cuando escribes una URL en el navegador?"
> 1. El navegador busca la IP del dominio via **DNS**
> 2. Establece conexion **TCP** con el servidor (3-way handshake)
> 3. Si es HTTPS, negocia **TLS/SSL** (certificado, encriptacion)
> 4. Envia peticion **HTTP** (GET / HTTP/1.1)
> 5. El servidor procesa y devuelve la **respuesta** (HTML, JSON...)
> 6. El navegador **renderiza** el contenido (parsea HTML, aplica CSS, ejecuta JS)

Concepto	Que es
DNS	Traduce dominio (google.com) a IP (142.250.184.14)
TCP	Protocolo de transporte fiable (garantiza entrega)
HTTP/HTTPS	Protocolo de aplicacion para transferir datos web
Puerto	Numero que identifica un servicio (80=HTTP, 443=HTTPS, 5432=PostgreSQL)
IP	Direccion unica de un dispositivo en la red
Firewall	Filtra trafico de red segun reglas
Proxy / Reverse Proxy	Intermediario. Nginx como reverse proxy recibe peticiones y las redirige a tu app

9.4 Nginx basico

```
# Reverse proxy basico
server {
    listen 80;
    server_name miapp.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

10. Git

10.1 Comandos esenciales

```
git init                        # iniciar repositorio
git clone url                   # clonar repo remoto
git status                      # ver estado actual
git add file.js                 # agregar archivo al staging
git add .                       # agregar todo
git commit -m "mensaje"         # crear commit
git push origin main            # subir al remoto
git pull origin main            # bajar cambios del remoto
git log --oneline               # ver historial resumido
git diff                        # ver cambios no staged
```

10.2 Ramas (branches)

```
git branch                      # ver ramas locales
git branch feature/login        # crear rama
git checkout feature/login      # cambiar a rama
git checkout -b feature/login   # crear y cambiar en un paso
git merge feature/login         # fusionar rama a la actual
git branch -d feature/login     # eliminar rama
```

10.3 Git Flow

Pregunta: "Que flujo de Git usas?"
- **main/master**: codigo en produccion, siempre estable
- **develop**: rama de desarrollo, aqui se integran features
- **feature/xxx**: ramas para cada funcionalidad nueva
- **hotfix/xxx**: parches urgentes directos a main

Cada feature se desarrolla en su rama, se hace PR, code review, y se mergea.

10.4 Conflictos

Pregunta: "Como resuelves un conflicto en Git?"
1. Git te marca las zonas en conflicto con `<<<<<<<` y `>>>>>>>`
2. Abres el archivo, eliges que codigo mantener (o combinas ambos)
3. Eliminas los marcadores de conflicto
4. `git add .` y `git commit`

10.5 Merge vs Rebase

Merge	Rebase
Crea un commit de merge	Reescribe el historial (lineal)
Historial no lineal pero completo	Historial limpio y lineal
Seguro en ramas compartidas	NUNCA en ramas compartidas/publicas

11. Conceptos Clave de Arquitectura

11.1 CI/CD

Pregunta: "Que es CI/CD?"
CI (Continuous Integration): cada vez que alguien sube codigo, se ejecutan tests automaticos y se compila. Si algo falla, se avisa inmediatamente.
CD (Continuous Deployment/Delivery): si los tests pasan, el codigo se despliega automaticamente a produccion (deployment) o queda listo para desplegar manualmente (delivery).

11.2 Monolito vs Microservicios

Monolito	Microservicios
Todo en una aplicacion	Servicios independientes
Facil de desarrollar al inicio	Complejo pero escalable
Un deploy, un repositorio	Cada servicio tiene su deploy
Si falla, falla todo	Fallos aislados

Tu respuesta: "Para proyectos pequenos/medianos empiezo con monolito. Solo separo en microservicios cuando hay una necesidad real de escalar partes independientemente."

11.3 Cache

Pregunta: "Que es cache y donde la usas?"
Almacenar datos frecuentemente accedidos en un lugar rapido para evitar recalcular o hacer queries repetidas.
- **Browser cache**: archivos estaticos (CSS, JS, imagenes)
- **CDN cache**: Cloudflare cachea contenido cerca del usuario
- **Redis**: cache en memoria para datos de la app (sesiones, queries frecuentes)
- **React**: useMemo, useCallback para cache en componentes

11.4 Variables de entorno

```
# .env (NUNCA se sube a Git)
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
JWT_SECRET=mi-secreto-super-largo
API_KEY=abc123

# Acceder en codigo
process.env.DATABASE_URL
```

11.5 HTTPS y SSL/TLS

Pregunta: "Como funciona HTTPS?"
1. El cliente se conecta al servidor y pide el certificado SSL
2. El servidor envia su certificado (emitido por una CA como Let's Encrypt)
3. El cliente verifica que el certificado es valido
4. Ambos negocian una clave simetrica para encriptar la comunicacion
5. A partir de ahi, todo el trafico va encriptado

11.6 WebSockets vs HTTP

HTTP	WebSocket
Request-Response (cliente pide, servidor responde)	Bidireccional (ambos envian cuando quieren)
Conexion se cierra despues de cada peticion	Conexion permanente abierta
APIs REST, paginas web	Chat en tiempo real, notifications, gaming

12. Preguntas de Comportamiento

Muchas entrevistas tienen una parte no tecnica. Prepara respuestas para estas usando el metodo **STAR**: Situacion, Tarea, Accion, Resultado.

Preguntas que te van a hacer (y como responder con TUS proyectos)

"Cuentame sobre ti"
Soy desarrollador full-stack y DevOps. Llevo anos trabajando en tecnologia, desde administrar servidores hasta construir aplicaciones web completas. Actualmente trabajo como tecnico informatico en la Comunidad de Madrid a traves de Ayesa, y en paralelo soy socio en Hytalia Network, donde dirijo la operacion tecnica. He construido varios productos en produccion como Finesse, una app de finanzas, y StackWatch, un sistema de monitorizacion. Me motiva construir cosas que funcionen bien y no se rompan.

"Cual es tu mayor fortaleza?"
Mi capacidad de hacer que las cosas funcionen end-to-end. No solo se hacer frontend o backend, sino que puedo montar la infraestructura, el CI/CD, el monitoring, y asegurarme de que todo esta en produccion y estable. Soy muy bueno resolviendo problemas porque he gestionado servidores con clientes reales que dependen del uptime.

"Cual es tu mayor debilidad?"
A veces quiero resolver los problemas demasiado rapido y no documento lo suficiente en el proceso. Es algo que estoy mejorando activamente, especialmente ahora que trabajo en equipo y la documentacion es clave para que otros entiendan lo que hice.

"Cuentame un proyecto del que estes orgulloso"
VarynHost. Cofunde un servicio de hosting con mas de 60 servidores activos y 60 clientes. Automatice el provisioning, monte los paneles custom y gestionaba el soporte. Lo que mas me enorgullece es que era infraestructura real con gente pagando y dependiendo de que no se cayera. Aprendi muchisimo sobre operaciones, automatizacion y responsabilidad sobre sistemas en produccion.

"Como manejas la presion o los deadlines?"
Priorizo. Identifico que es critico y que puede esperar. En VarynHost, cuando habia incidencias con servidores de clientes, tenia que resolver rapido porque habia gente pagando. Aprendi a mantener la calma, diagnosticar el problema, y comunicar al cliente mientras lo resolvia.

"Por que quieres trabajar aqui?"
[Investiga la empresa antes] Me interesa porque [algo especifico de la empresa: su producto, su stack, su cultura]. Creo que puedo aportar valor con mi experiencia en [X] y a la vez aprender y crecer en [Y].

"Donde te ves en 5 anos?"
Quiero seguir construyendo producto, idealmente en un rol mas senior donde pueda tomar decisiones de arquitectura y mentorizar a otros. Me interesa especialmente la interseccion entre desarrollo y operaciones, haciendo que los equipos puedan entregar mas rapido y con mas calidad.

"Como trabajas en equipo?"
Actualmente en Hytalia coordino con equipos en US y Espana, asi que estoy acostumbrado a comunicacion asincrona, definir responsabilidades claras y documentar decisiones. Creo que un buen equipo se basa en comunicacion clara y en que cada uno pueda trabajar de forma autonoma pero alineada.

13. Como Vender tus Proyectos en la Entrevista

> **Regla clave:** Habla de PROBLEMAS que resolviste y RESULTADOS, no de tecnologias. Al recruiter no le importa que usaste Docker. Le importa POR QUE lo usaste y QUE consiguio.

Finesse

No digas: "Hice una app con Next.js, React, TypeScript, Supabase y Tailwind"

Di: "Vi que Splitwise tenia limitaciones para mis necesidades, asi que construi mi propia alternativa. Es una PWA instalable que funciona offline, importa datos de CSV, y tiene un asistente financiero con IA. La hice full-stack, desde la autenticacion con OAuth hasta el Service Worker para modo offline."

StackWatch

Di: "Necesitaba monitorizar mis servidores de forma fiable. Construi un sistema que hace health checks cada 60 segundos, gestiona incidentes, envia alertas y tiene una status page publica. Todo el pipeline de CI/CD esta automatizado con Docker Swarm."

VarynHost

Di: "Cofunde un negocio de hosting que llego a tener mas de 60 clientes y 60 servidores activos en la UE. Automatice todo el provisioning, los paneles de control y la facturacion. Lo importante es que era infraestructura real con clientes pagando, asi que el uptime y la fiabilidad eran criticos."

Ayesa / Comunidad de Madrid

Di: "Trabajo como tecnico informatico dando soporte a empleados y directivos de las consejerias de la Comunidad de Madrid. Soy el enlace entre el proveedor tecnologico y la gerencia de Madrid Digital. Gestiono incidencias, diagnostico problemas y me aseguro de que se resuelvan dentro del SLA."

Hytalia Network

Di: "Soy socio en una joint venture entre una empresa americana y una espanola. Mi responsabilidad es la operacion diaria de la plataforma, definir el roadmap de producto, y coordinar con los otros socios. Llevo la parte tecnica: infraestructura, automatizacion y continuidad del servicio."

14. AWS Cloud Esencial

El 90% de ofertas DevOps piden "conocimientos de cloud". AWS tiene ~60% del mercado. Esto es lo que necesitas saber.

14.1 Que es Cloud Computing

> **Pregunta:** "Que es cloud computing?"
> En vez de comprar y mantener servidores fisicos propios, alquilas recursos (servidores, almacenamiento, redes, bases de datos) a un proveedor como AWS, que los gestiona en sus data centers. Pagas solo por lo que usas. Puedes escalar en minutos en vez de semanas.

14.2 Los 3 modelos de servicio

Modelo	Que te dan	Tu gestionas	Ejemplo
IaaS (Infrastructure)	Servidor virtual, red, storage	SO, apps, datos, config	AWS EC2, tu VPS actual
PaaS (Platform)	Plataforma para ejecutar codigo	Solo tu codigo y datos	Heroku, AWS Elastic Beanstalk
SaaS (Software)	Aplicacion completa	Nada tecnico	Gmail, Slack, Notion

> **Tu experiencia:** Con VarynHost y Creatiums ya trabajabas con IaaS (servidores dedicados/VPS). AWS es lo mismo pero en la nube y con mas servicios automatizados.

14.3 Servicios AWS que DEBES conocer

Compute (Computacion)

Servicio	Que es	Analogia con lo que ya sabes
EC2	Servidores virtuales (VPS en la nube)	Como los VPS que gestionabas en VarynHost
Lambda	Funciones serverless (ejecuta codigo sin servidor)	Como una API route de Next.js pero sin servidor
ECS / EKS	Ejecutar contenedores Docker en la nube	Como tu Docker Swarm pero gestionado por AWS

Storage (Almacenamiento)

Servicio	Que es	Para que
S3	Almacenamiento de objetos (archivos)	Imagenes, backups, archivos estaticos, logs
EBS	Disco duro virtual para EC2	El disco de tu servidor virtual

Base de datos

Servicio	Que es	Analogia
RDS	Base de datos relacional gestionada	Tu PostgreSQL pero AWS gestiona backups, updates, replicas
DynamoDB	Base de datos NoSQL serverless	Como MongoDB pero de AWS

Redes y seguridad

Servicio	Que es	Analogia
VPC	Red virtual privada en la nube	Tu red interna pero en AWS

Route 53	Servicio DNS	Como tu config de Cloudflare DNS
CloudFront	CDN (Content Delivery Network)	Como Cloudflare CDN
IAM	Gestion de usuarios, roles y permisos	Quien puede hacer que en tu cuenta AWS
Security Groups	Firewall virtual para EC2	Como iptables pero visual

Otros importantes

Servicio	Que es
CloudWatch	Monitorizacion, logs y alertas (como tu StackWatch pero de AWS)
ECR	Registro de imagenes Docker (como Docker Hub pero privado en AWS)
SNS / SQS	Notificaciones y colas de mensajes

14.4 Conceptos clave para la certificacion

Regiones y Availability Zones
AWS tiene **regiones** en todo el mundo (eu-west-1 = Irlanda, eu-south-2 = Espana). Cada region tiene multiples **Availability Zones** (data centers independientes). Despliegas en multiples AZs para alta disponibilidad: si un data center se cae, los otros siguen funcionando.

Modelo de responsabilidad compartida
AWS se encarga de: seguridad fisica, hardware, red global, hypervisor.
Tu te encargas de: tu SO, tu app, tus datos, configuracion de seguridad (security groups, IAM), encriptacion.
Regla facil: AWS protege LA nube. Tu proteges lo que hay EN la nube.

Modelo de precios
- **On-Demand**: pagas por hora/segundo. Sin compromiso. Mas caro.
- **Reserved**: compromiso 1-3 anos. 30-70% descuento.
- **Spot**: capacidad sobrante de AWS. Hasta 90% descuento pero te pueden quitar la instancia.
- **Free Tier**: 12 meses gratis con limites (750h EC2 t2.micro, 5GB S3, etc.)

14.5 AWS Cloud Practitioner - Tips para el examen

- Son 65 preguntas de opcion multiple, 90 minutos
- Necesitas 70% para aprobar
- NO te piden saber usar la consola ni escribir codigo
- Te preguntan conceptos: "que servicio usarias para X?", "que modelo de precios es mejor para Y?"
- Recursos gratis: AWS Skill Builder, curso de freeCodeCamp en YouTube (~4h)
- El examen cuesta $100 USD y se hace online desde casa

Con tu experiencia: Ya entiendes servidores, redes, DNS, firewalls, Docker y monitoring. El 60% del examen son cosas que ya sabes pero con nombres de servicios AWS. Solo necesitas aprender la nomenclatura.

15. Terraform - Infraestructura como Codigo

Terraform permite definir infraestructura en archivos de texto y crearla automaticamente. Es como pasar de Docker manual a Docker Compose, pero para TODA tu infraestructura.

15.1 Que es Infrastructure as Code (IaC)

Pregunta: "Que es Infrastructure as Code?"
En vez de crear servidores, redes y bases de datos haciendo clic en un panel web, los defines en archivos de codigo.
Ventajas:
- **Reproducible**: puedes crear el mismo entorno 100 veces de forma identica
- **Versionado**: los archivos van en Git, ves quien cambio que y cuando
- **Automatizable**: un pipeline de CI/CD puede crear o destruir infraestructura
- **Documentacion viva**: el codigo ES la documentacion de tu infraestructura

15.2 Como funciona Terraform

Pregunta: "Explicame el flujo de Terraform"
1. **Write**: Escribes archivos `.tf` describiendo la infraestructura que quieres
2. **Plan**: `terraform plan` muestra QUE va a crear/cambiar/destruir (sin hacer nada todavia)
3. **Apply**: `terraform apply` ejecuta los cambios en el proveedor cloud
4. **State**: Terraform guarda el estado actual en un archivo `terraform.tfstate` para saber que existe y que no

15.3 Sintaxis basica (HCL)

```
# Decir que proveedor usamos
provider "aws" {
  region = "eu-west-1"    # Irlanda
}

# Crear un servidor EC2
resource "aws_instance" "mi_servidor" {
  ami           = "ami-0c55b159cbfafe1f0"  # Ubuntu
  instance_type = "t3.micro"                # Tipo (2 vCPU, 1GB RAM)

  tags = {
    Name = "servidor-web"
  }
}

# Crear un bucket S3 para archivos
resource "aws_s3_bucket" "backups" {
  bucket = "mi-empresa-backups"
}

# Crear una base de datos RDS
resource "aws_db_instance" "postgres" {
  engine         = "postgres"
  engine_version = "16"
  instance_class = "db.t3.micro"
  db_name        = "miapp"
  username       = "admin"
```

```
  password       = var.db_password    # variable, nunca hardcodear passwords
}
```

Analogia con Docker:
- `Dockerfile` = define como construir una imagen → `main.tf` = define que infraestructura crear
- `docker build` = construye la imagen → `terraform plan` = muestra que va a hacer
- `docker run` = ejecuta el contenedor → `terraform apply` = crea la infraestructura
- `docker-compose.yml` = multiples servicios → `main.tf` = multiples recursos

15.4 Variables y Outputs

```
# variables.tf - Definir variables
variable "environment" {
  description = "Entorno (dev, staging, prod)"
  type        = string
  default     = "dev"
}

variable "db_password" {
  description = "Password de la base de datos"
  type        = string
  sensitive   = true    # no se muestra en logs
}

# Usar variables en recursos
resource "aws_instance" "server" {
  instance_type = var.environment == "prod" ? "t3.medium" : "t3.micro"

  tags = {
    Environment = var.environment
  }
}

# outputs.tf - Mostrar informacion despues de crear
output "server_ip" {
  value = aws_instance.server.public_ip
}

output "db_endpoint" {
  value = aws_db_instance.postgres.endpoint
}
```

15.5 Comandos esenciales

```
terraform init       # Inicializa el proyecto, descarga providers
terraform plan       # Muestra que va a hacer (preview)
terraform apply      # Ejecuta los cambios (crea/modifica/destruye)
terraform destroy    # Destruye TODA la infraestructura
terraform fmt        # Formatea los archivos .tf
terraform validate   # Valida la sintaxis
terraform state list # Lista los recursos gestionados
```

15.6 State (Estado)

Pregunta: "Que es el state en Terraform?"
El archivo `terraform.tfstate` es donde Terraform guarda que recursos ha creado y su estado actual. Sin este archivo, Terraform no sabe que existe y intentaria crear todo de nuevo.

En equipo: El state se guarda en un backend remoto (S3 + DynamoDB) para que todos los miembros del equipo trabajen con el mismo estado. NUNCA se sube a Git.

15.7 Modulos

```
# Modulos = componentes reutilizables de infraestructura
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"  # modulo publico
  version = "5.0.0"

  name = "mi-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["eu-west-1a", "eu-west-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]
}
```

Analogia: Los modulos de Terraform son como las imagenes de Docker Hub: alguien ya hizo el trabajo duro y tu solo lo configuras con tus parametros.

15.8 Terraform vs Alternativas

Herramienta	Tipo	Cuando usarla
Terraform	IaC declarativo, multi-cloud	Crear infraestructura en cualquier cloud
Ansible	Configuracion de servidores	Instalar software, configurar SO dentro de un servidor
CloudFormation	IaC solo para AWS	Si solo usas AWS y no necesitas multi-cloud
Pulumi	IaC con lenguajes reales (JS, Python)	Si prefieres escribir infra en tu lenguaje favorito

Pregunta: "Por que Terraform y no CloudFormation?"
Terraform es **multi-cloud**: funciona con AWS, GCP, Azure, Cloudflare, y cientos de providers. CloudFormation solo funciona con AWS. Si cambias de cloud o usas multiples, con Terraform no reescribes nada. Ademas, la sintaxis HCL es mas legible que los JSON/YAML de CloudFormation.

15.9 Ejemplo real: tu stack completo en Terraform

```
# Esto es lo que harias para desplegar algo como StackWatch en AWS

# VPC (red privada)
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  name   = "stackwatch-vpc"
  cidr   = "10.0.0.0/16"
}
```

```
# Servidor EC2
resource "aws_instance" "app" {
  ami           = "ami-ubuntu-latest"
  instance_type = "t3.small"
  subnet_id     = module.vpc.public_subnets[0]

  user_data = <<-EOF
    #!/bin/bash
    apt-get update
    apt-get install -y docker.io docker-compose
    cd /opt/stackwatch
    docker-compose up -d
  EOF

  tags = { Name = "stackwatch-app" }
}

# Base de datos RDS PostgreSQL
resource "aws_db_instance" "db" {
  engine         = "postgres"
  instance_class = "db.t3.micro"
  db_name        = "stackwatch"
  username       = "admin"
  password       = var.db_password
}

# DNS
resource "aws_route53_record" "app" {
  zone_id = var.zone_id
  name    = "stackwatch.lucasvicente.es"
  type    = "A"
  records = [aws_instance.app.public_ip]
  ttl     = 300
}
```

Lo que necesitas para la entrevista: No te van a pedir que escribas Terraform de memoria. Necesitas saber QUE es, POR QUE se usa, y haber tocado los comandos basicos al menos una vez. Con saber explicar el flujo (write → plan → apply) y los conceptos (state, providers, modules) es suficiente para Junior DevOps.

16. 100+ Preguntas Frecuentes de Entrevista Tech

Practica respondiendo estas en voz alta. No memorices: entiende y explica con tus palabras.

JavaScript (15)

1. Diferencia entre `==` y `===`
2. Que es hoisting?
3. Diferencia entre `var`, `let` y `const`
4. Que es un closure? Da un ejemplo
5. Como funciona el Event Loop?
6. Que es una promesa? Cuales son sus estados?
7. Diferencia entre `async/await` y `.then()`
8. Que hace `Array.map()` vs `Array.forEach()`?
9. Que es destructuring?
10. Diferencia entre spread y rest operator
11. Que es `this` en JavaScript?
12. Que son los template literals?
13. Que es el optional chaining (`?.`)?
14. Diferencia entre `null` y `undefined`
15. Que son los modulos en JS? Diferencia entre `import/export` y `require/module.exports`

TypeScript (10)

16. Que ventajas tiene TypeScript sobre JavaScript?
17. Diferencia entre `interface` y `type`
18. Que son los generics? Para que sirven?
19. Que es un union type?
20. Que hace `Partial<T>`?
21. Que hace `Omit<T, K>`?
22. Que significa `as const`?
23. Que es un enum y cuando lo usas?
24. Que es `unknown` vs `any`?
25. Como tipas una funcion que recibe un callback?

React (20)

26. Que es React y como funciona el Virtual DOM?
27. Diferencia entre componente funcional y de clase
28. Que son los hooks? Nombra los mas importantes
29. Como funciona `useState`?
30. Para que sirve `useEffect`? Que es el array de dependencias?
31. Diferencia entre `useMemo` y `useCallback`
32. Que es `useRef` y cuando lo usas?
33. Que es Context API y cuando lo usas?
34. Diferencia entre props y state
35. Por que las keys son importantes en listas?
36. Que es un componente controlado vs no controlado?

37. Como manejas formularios en React?
38. Que es el lifting state up?
39. Que es un custom hook? Para que sirve?
40. Como haces renderizado condicional?
41. Que es React.memo y cuando lo usas?
42. Que son los error boundaries?
43. Como haces fetching de datos en React?
44. Que es Suspense en React?
45. Que es el patron "composition" en React?

Next.js (10)

46. Que es Next.js y que anade sobre React?
47. Diferencia entre App Router y Pages Router
48. Que es un Server Component vs Client Component?
49. Cuando usas `"use client"` ?
50. Diferencia entre SSR, SSG e ISR
51. Como creas API routes en Next.js?
52. Que es el file-based routing?
53. Que es un middleware en Next.js?
54. Como manejas el SEO en Next.js?
55. Que es el streaming en Next.js?

SQL / PostgreSQL (10)

56. Diferencia entre INNER JOIN y LEFT JOIN
57. Que es un indice? Cuando lo creas?
58. Que es una transaccion? Para que sirve?
59. Diferencia entre WHERE y HAVING
60. Que es normalizacion de base de datos?
61. Que son las relaciones 1:N y N:M?
62. Como hacias paginacion en SQL?
63. Que es una foreign key?
64. Diferencia entre DELETE y TRUNCATE
65. Que es un ORM y que ventajas/desventajas tiene?

Docker / DevOps (10)

66. Que es Docker y por que se usa?
67. Diferencia entre imagen y contenedor
68. Que es un Dockerfile? Explicame cada instruccion
69. Que es Docker Compose? Para que sirve?
70. Que es un volume en Docker?
71. Diferencia entre contenedores y VMs
72. Que es Docker Swarm?
73. Que es CI/CD? Que herramientas usas?
74. Que son las GitHub Actions?
75. Como despliegas una aplicacion en produccion?

Linux / Redes (10)

76. Como ves los procesos corriendo en Linux?
77. Como ves el uso de disco y memoria?
78. Que son los permisos en Linux? Que significa 755?
79. Que es un reverse proxy? Para que sirve?
80. Que pasa cuando escribes una URL en el navegador?
81. Que es DNS?
82. Diferencia entre HTTP y HTTPS
83. Que es un firewall?
84. Que es SSH y para que se usa?
85. Que es un certificado SSL?

Git (5)

86. Que es Git y por que es importante?
87. Diferencia entre merge y rebase
88. Como resuelves un conflicto en Git?
89. Que es un pull request?
90. Que flujo de trabajo de Git usas?

Arquitectura / General (10)

91. Que es una API REST? Principios basicos
92. Diferencia entre autenticacion y autorizacion
93. Que es JWT? Como funciona?
94. Que es CORS y como lo solucionas?
95. Diferencia entre monolito y microservicios
96. Que es cache? Donde la usas?
97. Que son las variables de entorno?
98. Que es WebSocket? Cuando lo usas en vez de HTTP?
99. Que es la inyeccion SQL y como la previenes?
100. Que es XSS y como lo previenes?

AWS / Cloud (10)

101. Que es cloud computing? Que ventajas tiene?
102. Diferencia entre IaaS, PaaS y SaaS
103. Que es EC2?
104. Que es S3 y para que se usa?
105. Que es RDS?
106. Que es una VPC?
107. Que es IAM en AWS?
108. Que son las Availability Zones?
109. Que es el modelo de responsabilidad compartida?
110. Diferencia entre On-Demand, Reserved y Spot instances

Terraform / IaC (5)

111. Que es Infrastructure as Code?
112. Que es Terraform y como funciona? (write, plan, apply)
113. Que es el state en Terraform?

14. Diferencia entre Terraform y Ansible
15. Que es un modulo en Terraform?

Sobre ti (bonus)

16. Cuentame sobre ti
17. Por que quieres trabajar aqui?
18. Cual es tu mayor fortaleza?
19. Cual es tu mayor debilidad?
20. Donde te ves en 5 anos?
21. Cuentame un proyecto del que estes orgulloso
22. Como manejas la presion?
23. Como trabajas en equipo?
24. Tienes alguna pregunta para nosotros?
25. Que salario esperas?

> **Para la pregunta 109 ("Tienes preguntas?"):** SIEMPRE di que si. Pregunta: "Como es el dia a dia del equipo?", "Que herramientas usais?", "Cuales son los mayores retos tecnicos ahora mismo?", "Como es el proceso de onboarding?"

Fin de la Guia

Preparado para Lucas Vicente — Marzo 2026

Imprime este PDF o subelo a NotebookLM para generar tests interactivos.
Usa el test de autoevaluacion en `test-autoevaluacion.html` para medir tu progreso.